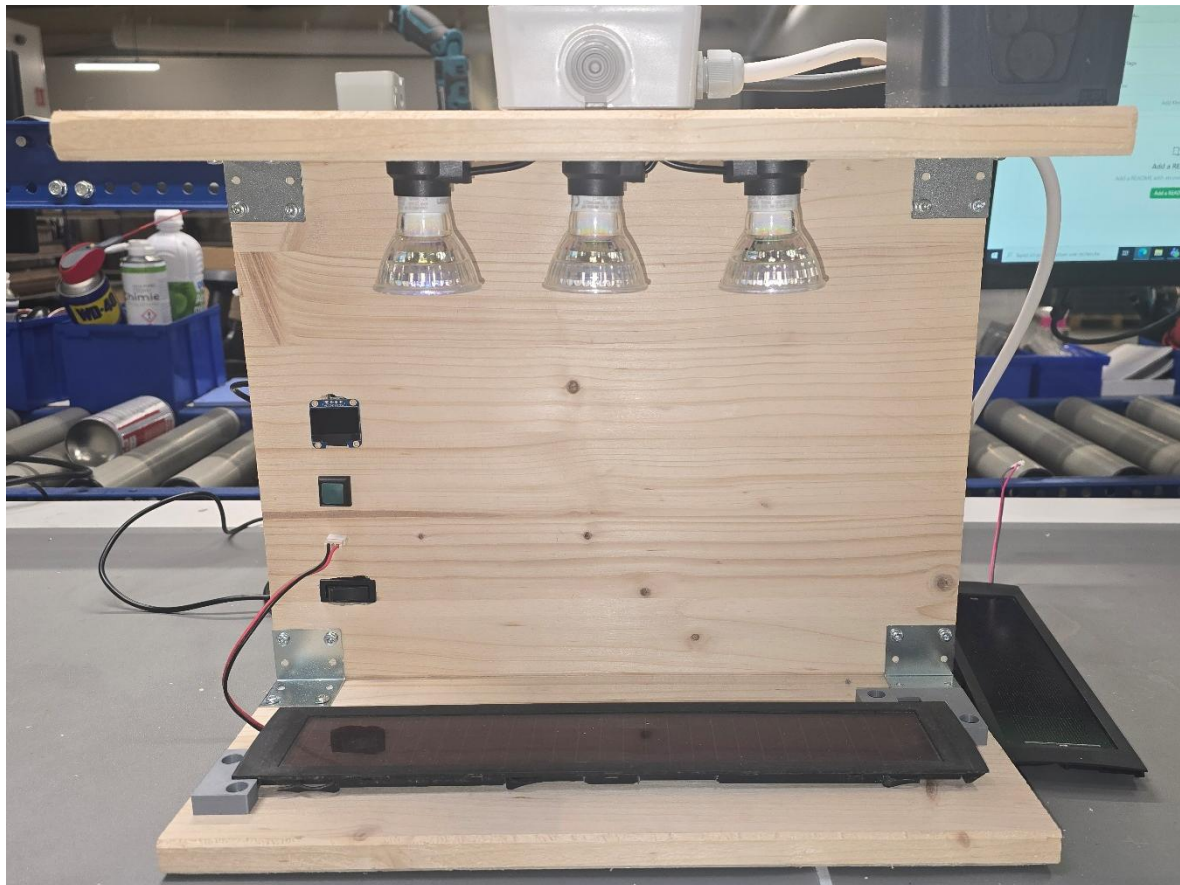


Dossier Technique

Projet Velux

MALINGE Louison

ST2ER



Sommaire

Situation dans le Projet	3
Présentation du projet	3
Description des tâches à réaliser	4
Contraintes diverses exprimées par le demandeur	5
Diagramme des Exigences	6
Spécifications fonctionnelles	7
Catalogue des acteurs	7
Diagramme Cas d'utilisation	7
Étude préliminaire	7
Prototypage	7
Réalisation du Projet	8
Diagramme	8
Diagramme de définition de bloc	8
Diagramme de bloc interne	8
Choix des composants	10
Schéma Structurel	12
Routage PCB	13
Communication Réseaux	14
Code	14
Test unitaire	19
Planification	20
Recette	20
Problèmes rencontrés	23
Conclusion	24
Annexe 1 : Planification Prévisionnelle	26
Annexe 2 : Planification réelle	27

Situation dans le Projet

Présentation du projet

Origine du projet

L'entreprise, pour donner suite à la demande d'un client, a le besoin de tester trois pièces différentes d'un volet roulant de la marque VELUX. Les trois pièces sont les suivantes : une batterie, une carte moteur et un panneau solaire. Les techniciens travaillant pour le client changeaient automatiquement ces trois pièces et les jetaient à la poubelle. Maintenant, ils préfèrent les tester pour récupérer les pièces encore fonctionnelles.

But du projet

Le but de ce projet est d'améliorer la ligne de test du panneau solaire pour permettre à l'opérateur de réaliser des mesures plus fiables et rapides afin d'évaluer si le panneau solaire peut encore être utilisé par le client VELUX. Le panneau solaire est une pièce très importante du volet roulant qui est sans fil, celui-ci permet de recharger la batterie. Le panneau solaire doit donc fonctionner été comme hiver et permettre une bonne recharge de la batterie même avec très peu de soleil.

Processus du projet

L'entreprise va recevoir 1200 pièces à tester de la part du client. Ces 1200 pièces vont être un test donné par le client pour gagner le marché.

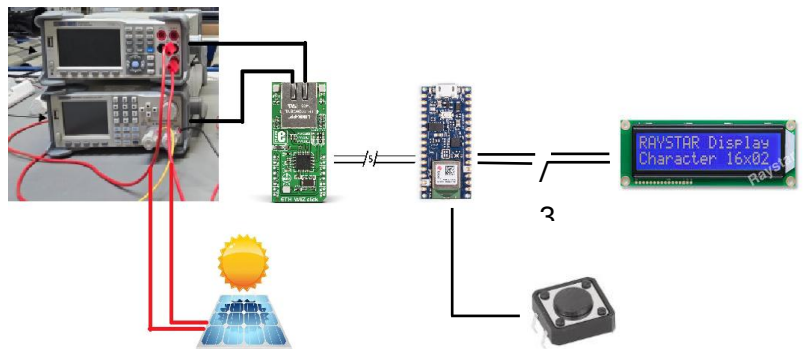
Objectif du projet

Afin d'éviter de jeter des pièces encore fonctionnelles, le projet vise à concevoir un système permettant de tester rapidement et de manière fiable les panneaux solaires, notamment en mesurant leur tension à vide, leur intensité de court-circuit et leur puissance. Un banc de test existe déjà pour effectuer ces mesures. Le banc de test est déjà composé des équipements de mesures (multimètre de table SIGLENT SDM3045x et charge électronique SIGLENT SDL1020x-e), d'un emplacement pour placer le panneau solaire et de lampes permettant de simuler le soleil. Le banc de test actuel doit être utilisé avec un opérateur pour effectuer les mesures en contrôlant les appareils de mesure manuellement. L'objectif du projet est donc d'améliorer le banc de test existant en automatisant les mesures via une Arduino nano.

Système de test

Le système sera composé de trois lampes pour simuler le soleil, d'une carte électronique avec un connecteur pour le panneau solaire afin de permettre de tester sa puissance. Le système communiquera en LAN grâce à un module Ethernet, avec :

- Un multimètre de table SIGLENT SDM3045x
- Une charge électronique SIGLENT SDL1020x-e.
-



Tâches à réaliser

Toutes les étapes du projet seront effectuées par moi-même étant seul à travailler sur le projet. Je recevrai de l'aide de Leo CASTRE (alternant au bureau des méthodes dans l'entreprise) et de Sean RAMAGET (ingénieur électronique)

Description des tâches à réaliser

L'objectif du projet est d'automatiser un banc de test déjà présent. Le test s'effectuait avec des mesures manuelles et qui étaient reportées dans un fichier Excel. Ce fichier Excel calculait le Fill Factor nécessaire pour connaître l'état du panneau solaire. Ma principale mission durant le projet sera donc d'améliorer ce banc de test pour rendre le test du panneau solaire plus rapide et fiable.

Le Fill Factor (FF) est un coefficient compris entre 0 et 1 qui se calcule grâce à la tension à vide (V_{oc} en volt), l'intensité de court-circuit (I_{sc} en mA) et la puissance maximum (P_{mpp} en mW). La formule est :

$$FF = \frac{P_{mpp}}{V_{oc} * I_{sc}}$$

Pour que le panneau solaire soit considéré en bon état, le Fill Factor doit être supérieur ou égal à 0,40 et la puissance supérieure ou égale à 55 mW.

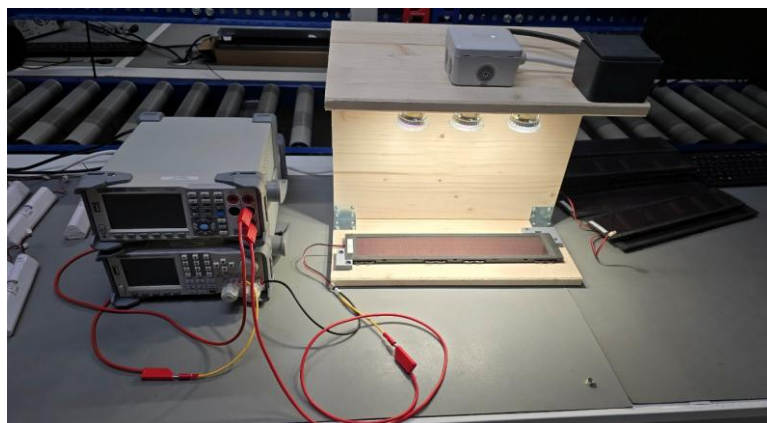


Figure 1 Ancien banc de test

Pour réaliser ce projet, j'ai donc commencé par utiliser le banc de test afin de me familiariser avec le test des panneaux solaires, avec la notion du Fill Factor... Puis les étapes du projet ont été :

- Réflexion sur la réalisation et choix des composants
- Prototypage sur breadboard
- Réalisation du schéma structurel et routage PCB
- Intégration du PCB et mise en place du nouveau banc de test automatisé.

Pour réaliser ces étapes, j'étais en entreprise où j'ai réalisé le PCB sur Kicad, le code sur Arduino IDE. Et pour les tests, j'ai utilisé le matériel présent en entreprise (multimètre, oscilloscope...)

Contraintes diverses exprimées par le demandeur

Les contraintes exprimées par le demandeur sont les suivantes :

- **La liste des choix techniques/technologiques**

Je devrais réaliser une liste expliquant chacun de mes choix, par exemple : pourquoi avoir choisi ce composant pour l'alimentation ? Pourquoi avoir placé l'écran ou le bouton à cet emplacement sur le banc de test ?

- **Le prototypage des structures électroniques et logicielles**

Je devrais concevoir un prototype sur breadboard afin de valider le fonctionnement du nouveau banc de test et de vérifier la communication entre l'Arduino Nano et les deux appareils de mesure.

- **Le schéma structurel sur KiCad**

KiCad est le logiciel utilisé dans l'entreprise, je devais donc l'utiliser pour réaliser le schéma structurel de mon projet.

- **Le routage sur KiCad**

Le routage sur KiCad devait répondre aux contraintes de compatibilité électromagnétique (CEM). Pour cela, le PCB sera réalisé sur quatre couches.

- **Le code du microcontrôleur**

Je devrais coder entièrement le programme de test du panneau solaire en m'assurant qu'il soit fiable et le plus rapide possible.

- **Le mode d'emploi/la procédure d'utilisation**

Je devais rédiger la procédure d'utilisation afin que n'importe quel opérateur puisse accéder au système sans formation spécifique.

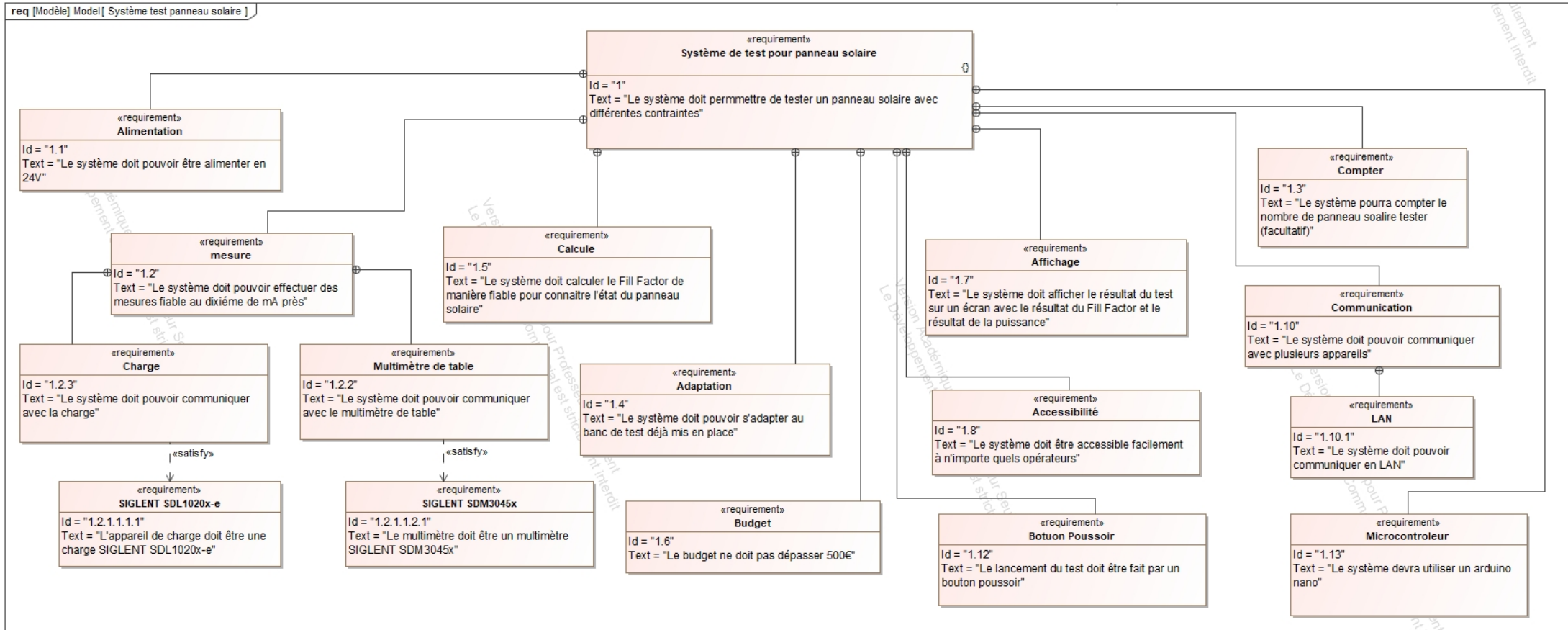
- **Les perspectives...**

Je devais envisager les améliorations à apporter au nouveau banc de test à la fin du projet afin de le rendre encore plus efficace.

- **Un coût de moins de 500 €**

Je devais ne pas dépasser un budget de 500 € afin que le nouveau banc de test soit le plus rentable possible pour l'entreprise.

Diagramme des Exigences

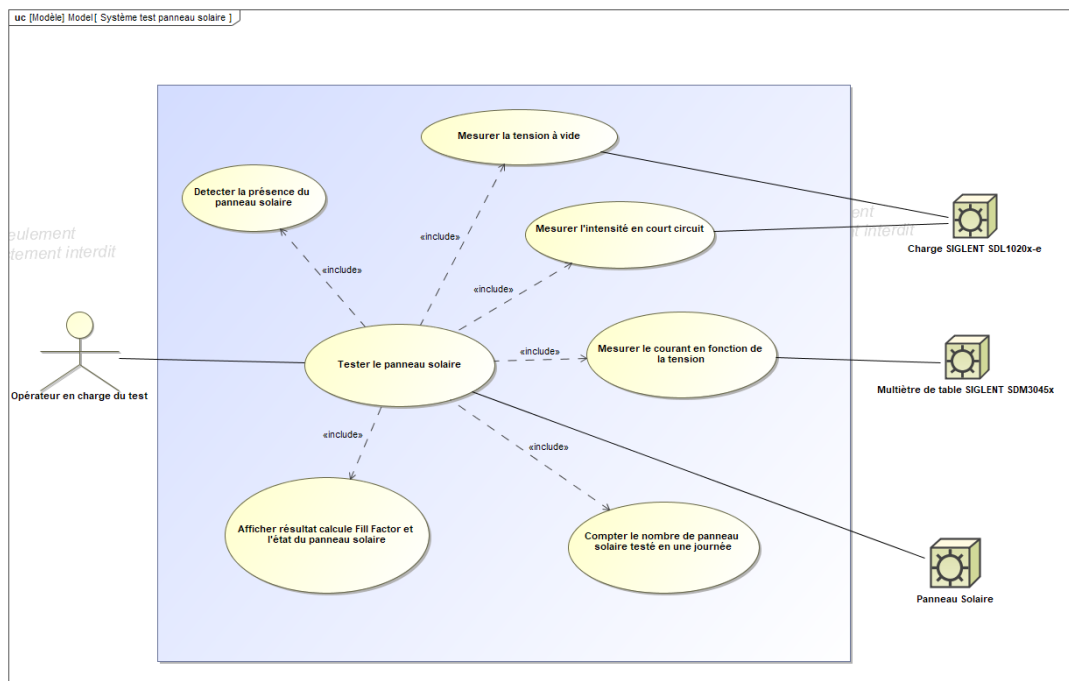


Spécifications fonctionnelles

Catalogue des acteurs

Acteur	Rôle
Opérateur de test	Effectuer le test du panneau solaire grâce au banc de test mis à sa disposition

Diagramme Cas d'utilisation



Étude préliminaire

Prototypage

Pour commencer le projet après avoir choisi mes composants, j'ai réalisé un prototypage sur une platine d'essai (breadboard) avec l'Arduino Nano 33 IoT, le module Eth Wiz Click et l'écran. Ce prototypage m'a permis de tester la fonctionnalité de la communication entre le microcontrôleur et les appareils de mesure SIGLENT. Pour cela, il a fallu que je programme l'Arduino en C++ afin d'envoyer des commandes SCPI (le langage des appareils SIGLENT). Ne sachant pas comment procéder au départ, j'ai utilisé l'intelligence artificielle pour m'aider, ce qui m'a permis de valider le bon fonctionnement des composants avant de lancer la fabrication du PCB.

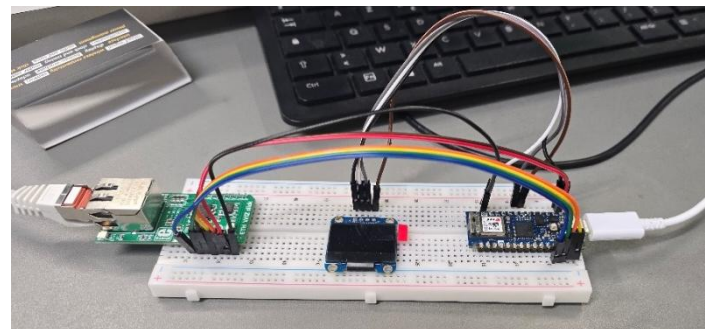
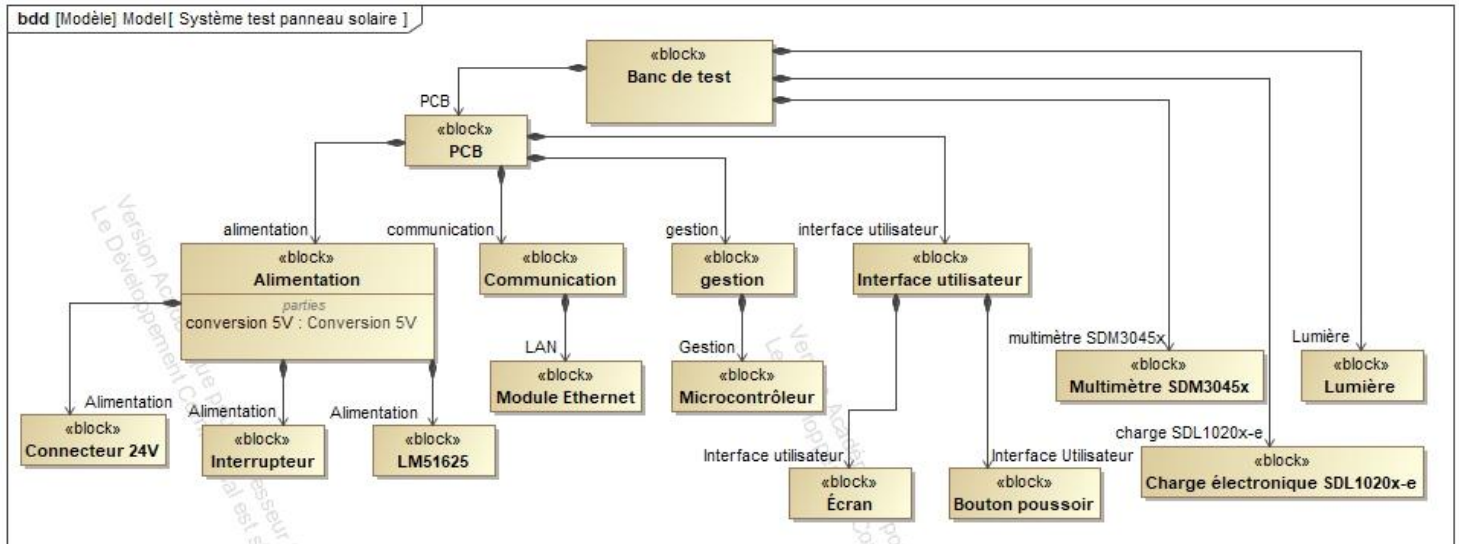


Figure 2 Prototypage sur breadboard

Réalisation du Projet

Diagramme

Diagramme de définition de bloc

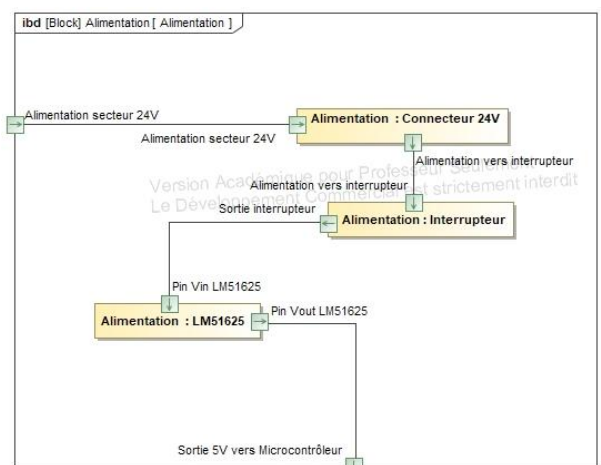


Ce diagramme permet de se rendre compte de quoi le banc de test va être composé. Il va être composé d'une partie alimentation pour convertir le 24V en 5V, d'une partie communication avec un module Ethernet pour communiquer avec la charge et le multimètre, d'une partie gestion avec l'Arduino Nano 33 IoT qui va permettre d'envoyer des commandes aux appareils de mesure via le module Ethernet et de gérer la partie interface utilisateur vers le bouton-poussoir pour lancer le test et de l'écran pour afficher le résultat du test.

Diagramme de bloc interne

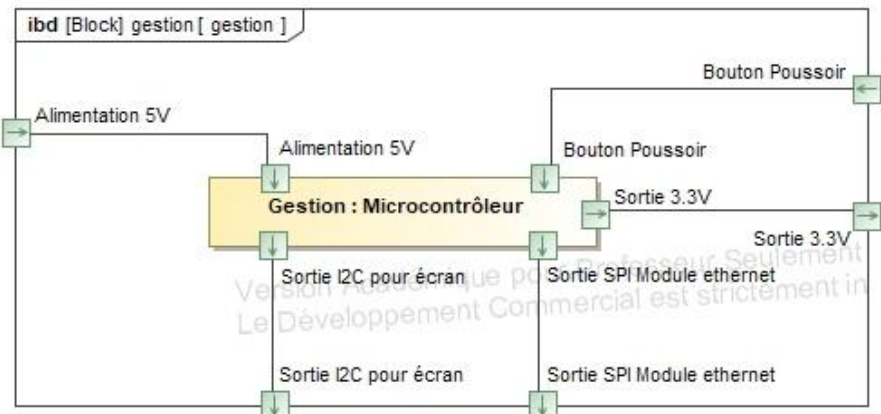
Alimentation

Ce diagramme permet de visualiser que l'alimentation de 24V va arriver dans le connecteur d'alimentation. Avant d'aller vers l'interrupteur et de l'interrupteur vers le LM51625.

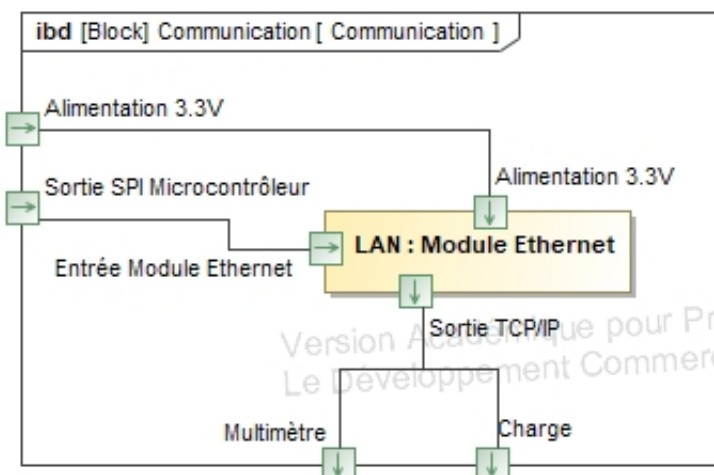


Gestion

Ce diagramme permet de visualiser que le microcontrôleur est alimenté en 5 V à la sortie du LM51625. Il convertit cette tension en 3,3 V pour alimenter l'écran, le module Ethernet et le bouton. Il communique avec l'écran via une interface I2C et avec le module Ethernet Eth Wiz Click via une interface SPI.



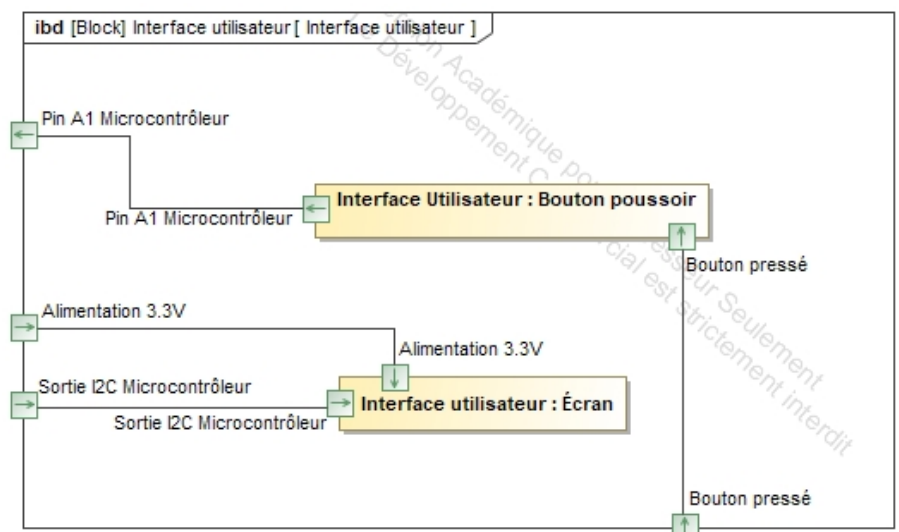
Communication



Ce diagramme permet de visualiser que le module Ethernet est alimenté en 3,3 V. Il communique avec le microcontrôleur via une interface SPI et avec la charge ainsi que le multimètre via le protocole TCP/IP.

Interface utilisateur

Ce diagramme permet de visualiser que l'utilisateur pourra appuyer sur le bouton poussoir relié au microcontrôleur et visualiser le résultat du test sur l'écran.



Choix des composants

Les composants choisis pour répondre à la demande de communication avec le multimètre et la charge sont une Arduino Nano 33 IoT, pour envoyer les commandes SCPI (Standard Commands for Programmable Instruments) nécessaires à la communication avec les appareils de mesure, et un module Ethernet Eth Wiz Click pour convertir la communication SPI de l'Arduino en protocole TCP/IP.

L'écran choisi est un écran OLED 128x64. Cet écran a été choisi pour sa facilité d'utilisation.

Le composant choisi pour effectuer la conversion est un LM51625 de la marque Texas Instruments. Ce composant a été choisi en raison de son rendement de 90 % et de sa faible dissipation thermique.

Le composant LM51625 est un régulateur de tension DC-DC abaisseur (buck converter). Pour ce faire, il a besoin de divers composants comme une bobine, des résistances et des condensateurs. Pour connaître leurs valeurs et leur câblage, il suffit de consulter la datasheet du LM51625.

8.2 Typical Application

The LMR51635 only requires a few external components to convert from a wide voltage range supply to a fixed output voltage. Figure 8-1 shows a basic schematic.

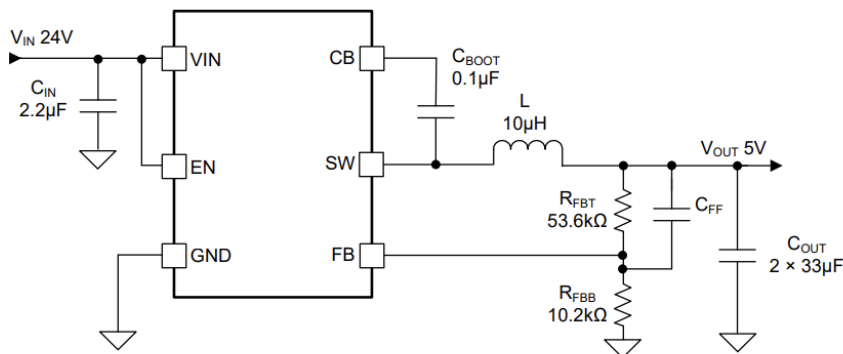


Figure 8-1. Application Circuit

The external components must fulfill the needs of the application and the stability criteria of the control loop of the device. Table 8-1 and Table 8-2 can be used to simplify the output filter component selection.

Table 8-1. Typical External Component Values for LMR51625, 2.5A Output Current

f_{sw} (kHz)	V_{OUT} (V)	L (µH)	$C_{OUT}^{(1)}$	R_{FBT} (kΩ)	R_{FBB} (kΩ)	C_{FF}
400	3.3	10	$2 \times 33\mu F / 10V$	31.6	10.2	Open
	5	15	$2 \times 22\mu F / 16V$	53.6	10.2	Open
	12	22	$22\mu F / 35V$	143	10.2	Open
1100	3.3	3.3	$33\mu F / 10V$	31.6	10.2	Open
	5	4.7	$22\mu F / 16V$	53.6	10.2	Open

(1) Ceramic capacitor is used in this table. All the C_{OUT} values are after derating.

Figure 3 Extrait de la datasheet du LM51625

J'ai choisi d'effectuer une conversion de 24 volts en 5 volts avec un courant de sortie de 2,5 ampères et une fréquence de 1100 kHz. Le 5 volts a été choisi pour alimenter l'Arduino Nano 33 IoT.

Les valeurs sont indiquées dans la datasheet, mais je les ai vérifiées en effectuant les calculs :

Calcul Bobine

$$L_{min} = \frac{V_{in_max} - V_{out}}{I_{out} \times K_{ind}} \times \frac{V_{out}}{V_{in_max} \times f_{sw}}$$

$$\Delta i_L = \frac{V_{out} \times (V_{in_max} - V_{out})}{V_{in_max} \times L \times f_{sw}}$$

$$L_{min} = \frac{24 - 5}{2,5 \times 0,35} \times \frac{5}{24 \times 1\,100\,000}$$

$$\Delta i_L = \frac{5 \times (24 - 5)}{24 \times (4,7 \times 10^{-6}) \times (1,1 \times 10^6)}$$

$$L_{min} = 4,11 \mu H$$

$$\Delta i_L = 0,76 A$$

En effectuant les calculs, je trouve une bobine avec une valeur minimum de 4,11 μH cohérente avec les 4,7 μH choisies et un courant de bobine de 0,76 A.

Calcul Résistance

$$R_{FBT} = \frac{V_{out} - V_{ref}}{V_{ref}} \times R_{FBB}$$

$$R_{FBT} = \frac{5 - 0,8}{0,8} \times (10,2 \times 10^3)$$

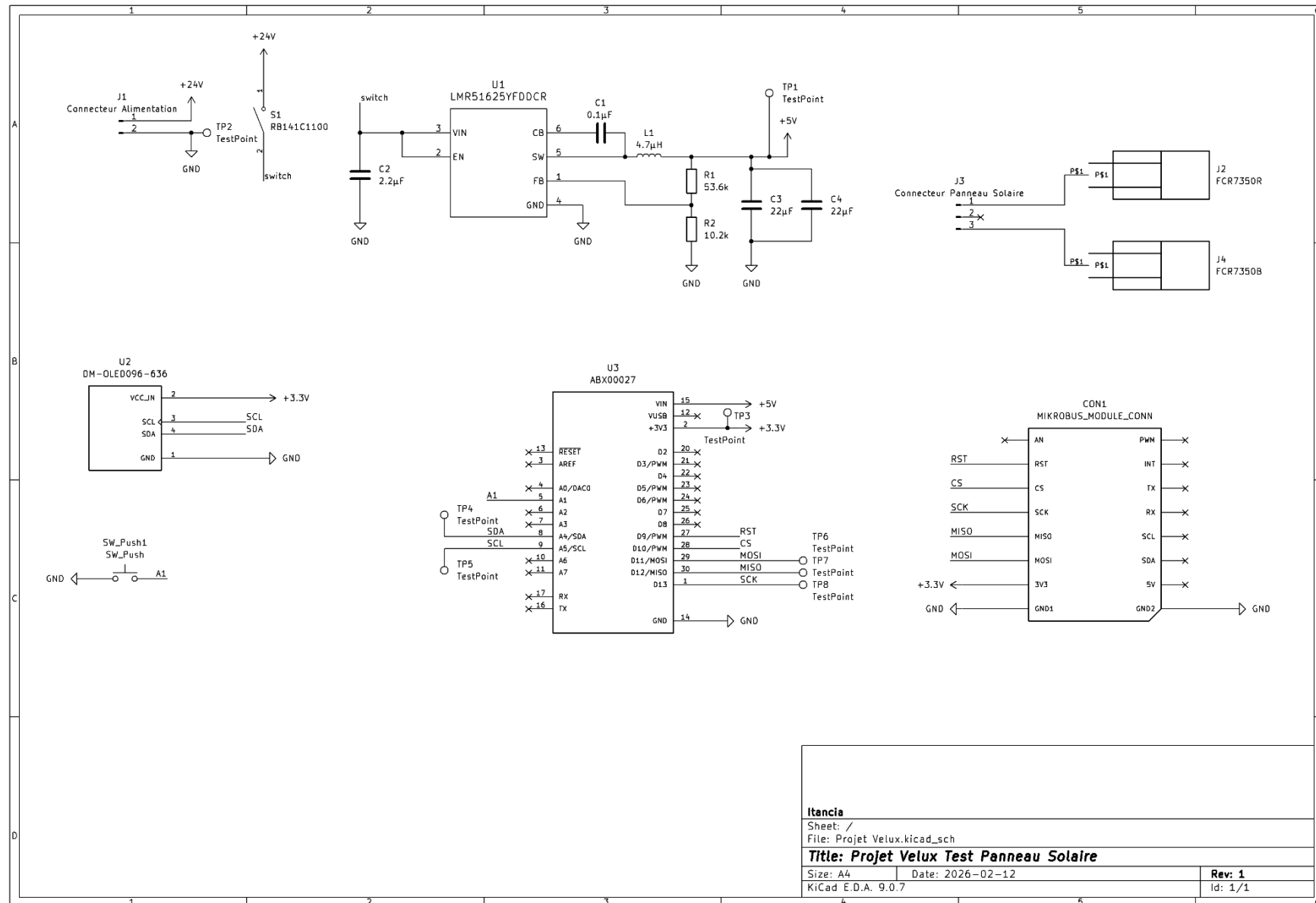
$$R_{FBT} = 53,55 k\Omega$$

En effectuant les calculs pour la résistance, je trouve une valeur RFTB de 53,55 k Ω , ce qui est parfaitement cohérent avec la résistance de 53,6 k Ω choisie. La résistance RFBB de 10,2 k Ω est une valeur conseillée dans la datasheet pour une tension de sortie de 5 V.

Voici donc un récapitulatif de tous les composants dont j'aurai besoin pour le schéma structurel et le routage du PCB présenté à la page suivante :

Arduino nano 33 iot	LM51625
Écran OLED 128*64	Condensateur 33uF/10 V x2
Eth Wiz Click	Bobine 10uH
Fiche banane 2 (rouge et noir)	Resistance 31,6kOhm
Connecteur 3 pin 5264 mk119	Résistance : 10,2 kOhm
Connecteur alimentation	Condensateur 2,2uF
Bouton poussoir	Condensateur 0,1uF
Écran	

Schéma Structurel



Itancia

Sheet: /
File: Projet Velux.kicad_sch

Title: Projet Velux Test Panneau Solaire

Size: A4 Date: 2026-02-12
KiCad E.D.A. 9.0.7

Rev: 1
Id: 1/1

Routage PCB

Le PCB a été réalisé sur quatre couches : la première couche sert à relier les composants CMS entre eux ; la deuxième couche est dédiée aux plans d'alimentation 3,3 V et 24 V ; la troisième couche est utilisée pour les signaux I2C et SPI ; enfin, la dernière couche est consacrée au plan de masse.

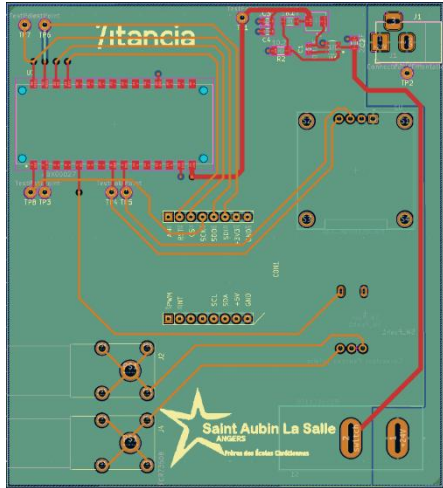


Figure 5 PCB router vu avec des 4 couches

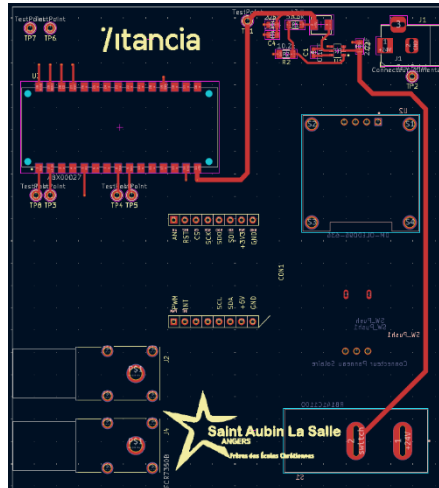


Figure 6 PCB router vu de la couche 1

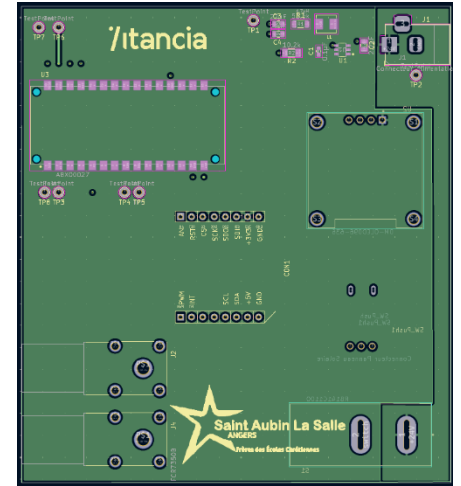


Figure 7 PCB router vu de la couche 2

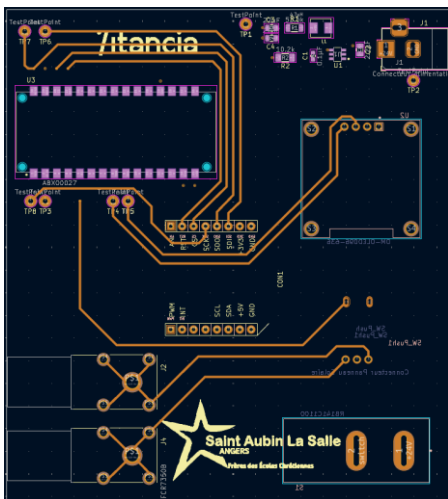


Figure 8 PCB router vu de la couche 3

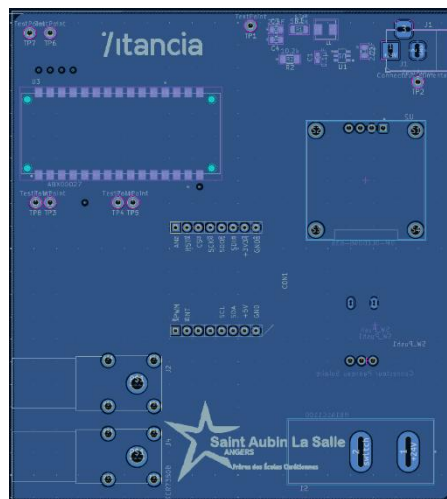


Figure 9 PCB router vu de la couche 4

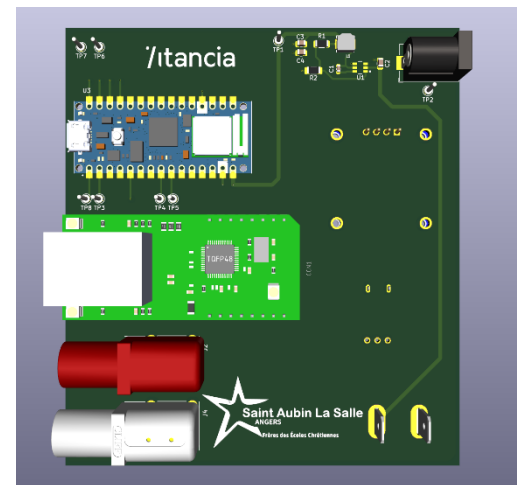


Figure 10 Vu 3D du PCB

Ce PCB a été réalisé sur quatre couches afin de répondre aux contraintes de compatibilité électromagnétique (CEM), et cela m'a été imposé par l'entreprise.

Cependant, celui-ci a été mal réalisé. Normalement, les signaux critiques (SPI, I²C) sont routés sur la première couche, le plan de masse sur la deuxième couche, le plan d'alimentation sur la troisième couche et les autres signaux sur la dernière couche.

Sur mon PCB, les composants CMS ont été placés sur la première couche et les signaux critiques ont été placés sur la couche 3, entre le plan d'alimentation (couche 2) et le plan de masse (couche 4). Ceci est donc une erreur de conception.

Communication Réseaux

La communication entre l'Arduino Nano et les appareils de mesure SIGLENT se fait via un module Ethernet Eth Wiz Click, qui convertit les commandes SPI de l'Arduino Nano en protocole TCP/IP pour les appareils de mesure. Pour ce faire, la charge et le multimètre de table sont connectés à un switch via des câbles RJ45, ainsi que le module Ethernet Eth Wiz Click.

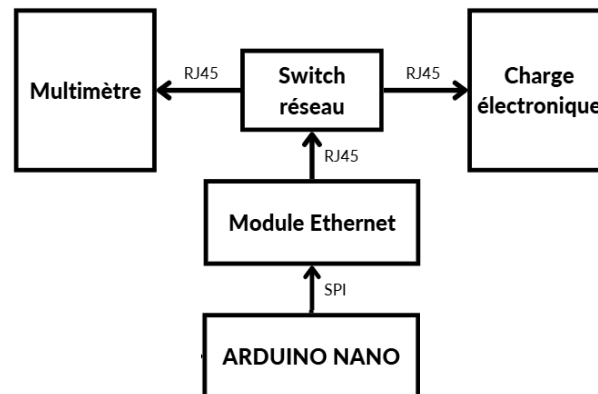


Figure 11 Schéma réseaux

Code

Pour la réalisation du programme, j'utilise les bibliothèques SPI.h pour la communication SPI avec le module Ethernet, Wire.h pour la communication I²C avec l'écran, et Ethernet.h pour la communication Ethernet avec les appareils de mesure SIGLENT.

Le code est réparti en plusieurs fichiers :

- Le fichier .ino contient l'inclusion des bibliothèques, la fonction setup() qui initialise les différents composants, ainsi que la fonction loop() qui permet de tester successivement plusieurs panneaux solaires.
- Les fichiers Ecran.h et Ecran.cpp regroupent toutes les fonctions liées à la gestion de l'écran.
- Les fichiers Siglent.h et Siglent.cpp contiennent les fonctions permettant de communiquer avec les deux appareils de mesure SIGLENT.
- Les fichiers Bouton.h et Bouton.cpp regroupent les fonctions liées au bouton poussoir.

Au démarrage du système, la fonction **setup()** appelle les différentes fonctions d'initialisation afin de mettre en place les communications SPI, Ethernet et I2C. Elle initialise également l'écran, vérifie la connexion avec la charge électronique en demandant son identification et en contrôlant la réponse reçue, puis teste la communication avec le multimètre.


```
/*=====SETUP pour initialiser tous les composants=====*/
void setup()
{
    init_com();    // Fonction pour initialiser les communications
    setup_ecran(); // Fonction pour initialiser l'écran
    delay(1000);   // Délai pour laisser le temps d'initialiser le module Ethernet et l'écran

    affichage_test_charge(); // Fonction pour tester la connexion avec la charge
    delay(500);
    affichage_test_multi(); // Fonction pour tester la connexion avec le multimètre
    delay(500);
}
```

Figure 12 Fonction setup du programme

La fonction `loop()` affiche le message « Appuyer sur le bouton » tant que le bouton poussoir n'est pas actionné. Lorsqu'un appui est détecté, le test est lancé. Une fois le test terminé, l'affichage revient au message d'attente.

```
/*=====LOOP pour appeler toutes les fonctions en boucle=====*/
void loop()
{
    bool Valeur_Bouton = traiterEtatBouton(); //Incrémenter la variable avec le résultat de la fonction pour connaître l'état du bouton

    if (Valeur_Bouton == 0) //Si la valeur du bouton est égal à 0 alors lancer le test
    {
        test_en_cours();    // Fonction pour afficher "Le test est en cours" et tester la connexion du panneau solaire
        afficher_Fill_Factor(); // Appeler la fonction qui affiche la valeur du Fill Factor
    }
    else // Sinon
    {
        affichage_debut(); // Appeler la fonction qui affiche le texte "Appuyer sur le bouton"
        delay(10);
    }
}
```

Figure 13 Fonction loop du programme

Le programme fonctionne de la manière suivante : lorsque le test démarre, le message « Le test est en cours » est affiché à l'écran. Ensuite, la fonction `afficher_Fill_Factor()` est appelée afin d'afficher le résultat du Fill Factor.

Pour obtenir cette valeur, la fonction `calculer_Fill_Factor()` est utilisée. Son calcul repose sur les mesures suivantes :

- La tension à vide (Voc) ;
- L'intensité de court-circuit (Isc) ;
- La puissance maximale (Pmpp).

Ces différentes valeurs sont récupérées à l'aide des fonctions de mesure correspondantes.

```
/*=====Fonction pour mesurer la tension à vide=====*/
float mesure_voc()
{
    set_charge_state(clientCharge, siglentCharge, port, "SOUR:FUNC VOLT"); // Mode tension continue
    set_charge_state(clientCharge, siglentCharge, port, "SOUR:INP OFF"); // Désactiver mode imposer tension
    delay(500);
    String Voc = interroger_instrument(clientCharge, siglentCharge, port, "MEAS:VOLT?"); // Lire tension
    float Tension_a_vide = Voc.toFloat(); // Convertir en float
    delay(100);
    return Tension_a_vide; // Retourner la valeur
}
```

Figure 14 Fonction pour mesurer la tension à vide

```
/*=====Fonction pour mesurer l'intensité de court-circuit=====*/
float mesure_isc()
{
    delay(2000);
    set_charge_state(clientCharge, siglentCharge, port, "SOUR:INP ON"); // Activer mode imposer tension
    delay(1000);
    set_charge_state(clientCharge, siglentCharge, port, ":SOURce:SHORt:StAtE ON"); // Activer mode court-circuit
    delay(1000);

    interroger_instrument(clientMulti, siglentMulti, port, "MEAS:CURRE:DC?"); // Lire intensité
    delay(1000);
    String Isc = interroger_instrument(clientMulti, siglentMulti, port, "MEAS:CURRE:DC?"); // Lire intensité
    delay(1000);

    float Intensite_de_cc = Isc.toFloat(); // Convertir en float
    delay(100);

    set_charge_state(clientCharge, siglentCharge, port, ":SOURce:SHORt:StAtE OFF"); // Désactiver mode court-circuit
    float courant_isc_mA = (Intensite_de_cc * 1000); // Conversion en mA
    return courant_isc_mA;
}
```

Figure 15 Fonction pour mesurer l'intensité de court-circuit

```
/*=====Fonction pour mesurer/calculer la puissance en mW=====*/
float mesure_Pmpp()
{
    float Pmpp_Max = 0; // Puissance maximale
    float Pmpp_actuel = 0; // Puissance actuelle

    set_charge_state(clientCharge, siglentCharge, port, "SOUR:FUNC VOLT"); // Mode tension continue
    set_charge_state(clientCharge, siglentCharge, port, "SOUR:INP ON"); // Activer imposer tension

    for (float vtest = 0.5; vtest < 30.0; vtest += 0.5) // Balayage tension
    {
        set_charge_state(clientCharge, siglentCharge, port, "SOUR:VOLT:LEV " + String(vtest)); // Imposer tension
        delay(250);

        String intensite = interroger_instrument(clientMulti, siglentMulti, port, "MEAS:CURRE:DC?"); // Lire courant
        float courant = intensite.toFloat();
        float courant_mA = courant * 1000;
        float Pmpp_actuel = vtest * courant; // Calcul puissance
        delay(100);

        if (Pmpp_actuel > Pmpp_Max) // Si puissance actuelle > puissance max
        {
            Pmpp_Max = Pmpp_actuel;
            delay(100);
        }
        else if (Pmpp_actuel < Pmpp_Max) // Sinon si puissance actuelle < puissance max
        {
            break; // Arrêter boucle
        }
    }

    float Pmpp_Max_mW = Pmpp_Max * 1000; // Conversion en mW
    return Pmpp_Max_mW;
}
```

Figure 16 Fonction pour mesurer le Puissance maximum

Les fonctions `set_charge_state()` et `interroger_instrument()` sont déclarées dans le fichier `Siglent.cpp`, tout comme les tableaux contenant les adresses IP, les ports de communication et les clients Ethernet.

La fonction `set_charge_state()` permet de configurer la charge électronique, par exemple pour réaliser une mesure de tension à vide. La fonction `interroger_instrument()` permet quant à elle d'envoyer une commande SCPI à un appareil de mesure afin de récupérer une valeur mesurée.

Le paramètre `EthernetClient &client` permet de sélectionner l'appareil avec lequel la communication doit être établie (charge électronique ou multimètre).

Le paramètre `IPAddress ip` permet de définir l'adresse IP du périphérique auquel la commande est envoyée.

Le paramètre `int port` permet de spécifier le port de communication utilisé, ici le port 5025.

Enfin, le paramètre `String cmd` contient la commande SCPI envoyée à l'appareil. Par exemple, la commande « MEAS:CURRENT:DC? » demande au multimètre la valeur du courant continu qu'il mesure.

```
/*=====Déclarer variables pour connexion Ethernet=====*/

#define eth_cs 10 // Définir le Chip Select sur la pin 10 de l'Arduino
#define eth_rst 9 // Définir le reset sur la pin 9 de l'Arduino

extern byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED }; // Définir l'adresse MAC de l'Arduino
extern IPAddress ip(192, 168, 1, 100); // Définir l'adresse IP de l'Arduino
extern IPAddress siglentCharge(192, 168, 1, 102); // Indiquer l'adresse IP de la charge
extern IPAddress siglentMulti(192, 168, 1, 101); // Indiquer l'adresse IP du multimètre
extern const int port = 5025; // Définir le port

EthernetClient clientCharge; // Déclarer la charge comme client
EthernetClient clientMulti; // Déclarer le multimètre comme client
```

Figure 17 Déclaration variable

```
/*=====Fonction pour interroger un appareil SIGLENT=====*/
String interroger_instrument(EthernetClient &client, IPAddress ip, int port, String commande)
{
    if (!client.connected()) client.connect(ip, port); // Si le client n'est pas connecté, on tente de se connecter
    if (client.connected()) // Si la connexion est établie
    {
        client.println(commande); // Envoyer la commande

        unsigned long start = millis(); // Mémoriser l'heure de départ

        while (!client.available() && millis() - start < 2000); // Attendre des données ou 2 secondes
        if (client.available()) // Si des données sont disponibles
        {
            String resp = client.readStringUntil('\n'); // Lire jusqu'au retour à la ligne
            resp.trim(); // Supprimer les espaces et retours superflus
            return resp; // Retourner la réponse
        }
    }
    return "Err"; // Si pas de connexion ou pas de réponse
}
```

Figure 18 Fonction pour interroger un instrument de mesure

```
/*=====Fonction pour configurer un appareil SIGLENT=====*/
void set_charge_state(EthernetClient &client, IPAddress ip, int port, String cmd)
{
    if (!client.connected()) client.connect(ip, port); // Si le client n'est pas connecté
    if (client.connected()) { // Si la connexion est établie
        client.println(cmd); // Envoyer la commande
        delay(150); // Délai pour traitement de la commande
    }
}
```

Figure 19 Fonction pour paramétrer un instrument de mesure

Test unitaire

Les fiches de test (voir plan de test) permettent de tester l'alimentation entière du PCB (GND, 24V, 5V, 3,3 V). Ces tests se sont parfaitement déroulés, avec une conversion en 5V du LM51625 pour l'Arduino Nano. Et la conversion du 3,3 V pour le module Ethernet et l'écran. Tout étant bien alimenté, j'ai pu procéder au test pour vérifier la communication avec la charge et le multimètre (voir figure 20)

Identification du Test		Ethernet		Référence du Module Testé :	Communication Ethernet		
Objectif du test :		Le but du test est de tester la communication Ethernet entre les appareils de mesure et le module Ethernet pour vérifier ainsi que les bonnes valeurs sont envoyées et que le test est fiable					
Nom du testeur :		Louison MALINGE		Date :	20/04/2026		
Moyens mis en œuvre :		Logiciel : Wireshark		Matériel : HUB Ethernet / PC		Outils de développement :	
Prérequis/Préconditions		Arduino programmé pour communication avec la charge et le multimètre					
Num étape	Procédure du test :		Résultat attendu		Critères de Réussite	Résultat obtenu	Validation (O/N)
1	➤ Brancher la charge, le multimètre, le module Ethernet et le PC au HUB Ethernet ➤ Ouvrir l'application Wireshark et suivre la procédure d'utilisation de Wireshark.		➤ Obtenir la communication entre la charge et le module Ethernet ➤ Obtenir la communication entre le multimètre et le module Ethernet		➤ Observer la communication entre les appareils de mesure et le module Ethernet	➤ J'ai obtenu la communication entre le module Ethernet et les appareils de mesure	O
Résultats	[X] OK Anomalie [] mineure [] majeure [] bloquante [] NOK					Réf. anomalie :	1
Conclusion du test :		Le test de la communication Ethernet est réussi					

Figure 20 Fiche de test pour communication en Ethernet

Afin d'effectuer le test avec le logiciel, j'ai rédigé une procédure pour suivre étape par étape l'utilisation de Wireshark pour décrypter les trames TCP envoyées entre l'Arduino et la charge et entre l'Arduino et le multimètre (voir Procédure de test Wireshark).

Une anomalie s'est révélée en testant plusieurs panneaux solaires d'affilée. Celui-ci était testé négativement à cause du multimètre qui renvoyait une donnée erronée ou une erreur « Err ». (Voir correction anomalie 1)

Planification

Sur la planification (voir annexe 1 et 2), un jour signifie une demi-journée, soit 3 h 20. On peut constater une différence entre le prévisionnel et le réel. J'ai terminé le projet avec 2 jours d'avance, car j'ai passé moins de temps que prévu sur la réalisation du schéma structurel et sur le routage du PCB. J'avais prévu 3 jours pour le schéma structurel et 3 jours pour le routage du PCB, que j'ai réalisé en 1 jour pour le schéma et 1 jour pour le routage.

J'ai aussi mis moins de temps pour la fabrication du PCB et son test : j'ai passé 2 jours pour réaliser ces deux tâches contre les 4 jours prévus. J'ai également mis 2 jours de moins pour intégrer le PCB au banc de test et tester entièrement le système.

Cependant, malgré tous ces jours d'avance, j'ai terminé le projet dans la planification initiale de seulement 2 jours, car j'ai passé plus de temps sur le code. Ce temps gagné m'a permis d'ajouter des options facultatives au projet, comme compter le nombre de panneaux solaires testés en une journée et détecter le branchement d'un panneau solaire.

Recette

Voici le prototype final du banc de test, parfaitement fonctionnel. Celui-ci a passé tous les tests. Certains tests de panneaux solaires ont échoué, ceci était dû à des erreurs dans la réalisation du code, qui ont été corrigées et améliorées au fur et à mesure de l'écriture du code.

Vous retrouverez les différentes étapes ainsi que tous les fichiers ayant contribué à la réalisation du projet sur le Git : <https://github.com/louisonmalinge/Projet-Velux> .

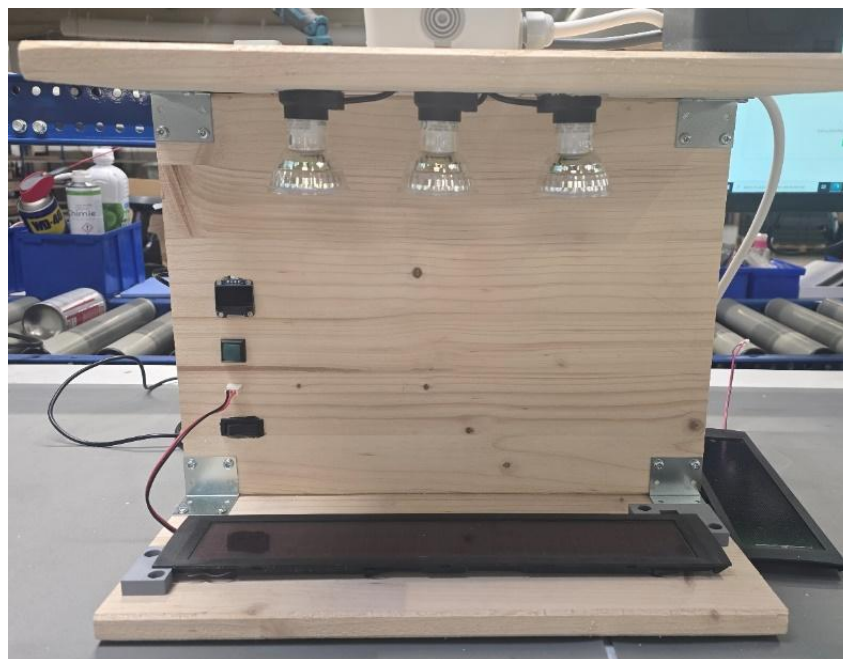


Figure 20 Banc de test automatisé et terminé

Voici les différents procès-verbaux des tests permettant de valider le fonctionnement du banc de test :

PT associé		GND	Réf. testé	Plan de masse
Type du test :		Structurel		
Objectif du test :		Vérifier le plan de masse		
Résumés des tests				
ID Test	Description du Test	Date	Résultats	Observations/Anomalies
T1	Tester GND Arduino	06/03/2026	Réussi	X
T2	Tester GND Ecran	06/03/2026	Réussi	X
T3	Tester GND module Ethernet	06/03/2026	Réussi	X
T4	Tester GND Condensateur C2	06/03/2026	Réussi	X
T5	Tester GND Condensateur C3	06/03/2026	Réussi	X
T6	Tester GND Condensateur C4	06/03/2026	Réussi	X
T7	Tester GND Résistance R2	06/03/2026	Réussi	X
Nature des modifications apportées au fichier source du module testé				
X				

PT associé		24V	Réf. testé	Test tension 24V
Type du test :		Fonctionnel		
Objectif du test :		Vérifier l'alimentation en 24V		
Résumés des tests				
ID Test	Description du Test	Date	Résultats	Observations/Anomalies
T1	Test 24V sortie connecteur d'alimentation	06/03/2026	Réussi	X
T2	Test 24V entrée du LM51625 via condensateur C2	06/03/2026	Réussi	X
Nature des modifications apportées au fichier source du module testé				
X				

PT associé		5V	Réf. testé	Test tension 5V
Type du test :		Fonctionnel		
Objectif du test :		Vérifier la conversion 24V en 5V		
Résumés des tests				
ID Test	Description du Test	Date	Résultats	Observations/Anomalies
T1	Test 5V sur le TP1	06/03/2026	Réussi	Conversion parfaite en 5V
Nature des modifications apportées au fichier source du module testé				
X				

PT associé		3,3V	Réf. testé	Test tension 3,3V
Type du test :		Fonctionnel		
Objectif du test :		Vérifier la conversion 5V en 3,3V et le plan d'alimentation		
Résumés des tests				
ID Test	Description du Test	Date	Résultats	Observations/Anomalies
T1	Test 3,3V sur le TP3	06/03/2026	Réussi	Conversion parfaite en 3,3V
T2	Test 3,3V sur l'écran	06/03/2026	Réussi	Écran alimenté en 3,3V
T3	Test 3,3V sur le module Ethernet	06/03/2026	Réussi	Module Ethernet alimenté en 3,3V
Nature des modifications apportées au fichier source du module testé				
X				

PT associé		Ethernet	Réf. testé	Ethernet
Type du test :		Intégration		
Objectif du test :		Le but du test est de tester la communication Ethernet entre les appareils de mesure et le module Ethernet		
Résumés des tests				
ID Test	Description du Test	Date	Résultats	Observations/Anomalies
T1	Tester communication Ethernet	21/04/2026	Réussi	X
Nature des modifications apportées au fichier source du module testé				
Voir modification dans le rapport correction d'anomalie				

PT associé		Ecran	Réf. testé	Ecran
Type du test :		Unitaire		
Objectif du test :		Le but du test est de tester le bon fonctionnement de l'écran		
Résumés des tests				
ID Test	Description du Test	Date	Résul-tats	Observations/Anoma-lies
T1	Teste ecran	21/04/2026	Réussi	X
Nature des modifications apportées au fichier source du module testé				
X				

Problèmes rencontrés

Cependant, malgré la réussite des tests, j'ai rencontré des problèmes au niveau de la conception du PCB.

Dans un premier temps, j'avais terminé le routage du PCB, mais celui-ci était trop grand. Il mesurait 10,5 cm de largeur, ce qui faisait passer son prix à 40 €. J'ai donc dû le réduire d'au moins 0,5 cm afin de ramener son coût de fabrication à 20 €.

Le deuxième problème concernait le module Ethernet Eth Wiz Click, que j'ai placé du mauvais côté du PCB lors du routage. L'empreinte du module Ethernet était du même côté que l'écran, alors qu'elle devait être du même côté que l'Arduino. Cela m'a obligé à le placer à l'envers sur le PCB (voir figure 21).

Un autre problème rencontré au niveau du routage est l'oubli des trous afin de visser le PCB au banc de test. Cet oubli m'a obligé à trouver une solution pour fixer le PCB au banc de test.

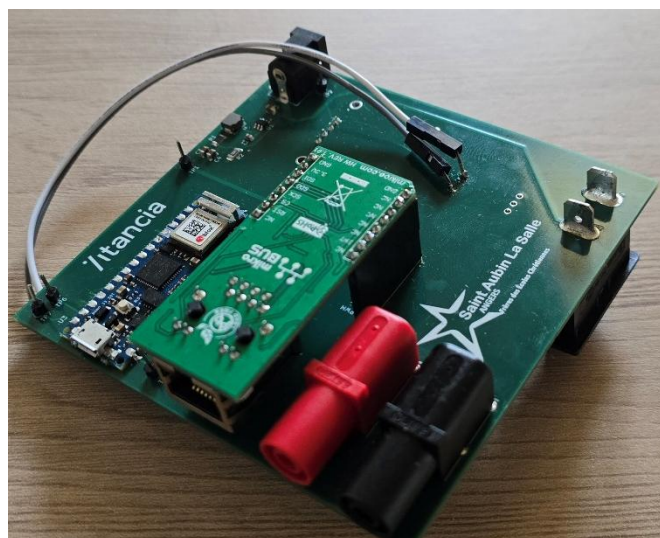


Figure 21 PCB final

J'ai rencontré un problème important au niveau du test du panneau solaire. Le test n'était pas fiable et variait en fonction du précédent test. Un panneau solaire testé positivement est testé négativement après le test d'un autre panneau solaire qui était testé négativement. Ce problème est dû à la mesure du courant de court-circuit qui était faussée ou inexistante. Soit le panneau solaire était testé négativement, soit la valeur du Fill Factor était supérieure à 1. Ce qui ne doit pas être possible. Ce problème se reproduisant systématiquement, j'ai dû le corriger en modifiant le code (voir correction anomalie).

Conclusion

Le projet a été mené à son terme avec une intégration du PCB au banc de test. Les tests d'intégration s'étant déjà bien passés avec la communication entre l'Arduino, la charge et le multimètre, il m'a suffi de fixer le PCB au banc de test et d'effectuer tous les branchements.

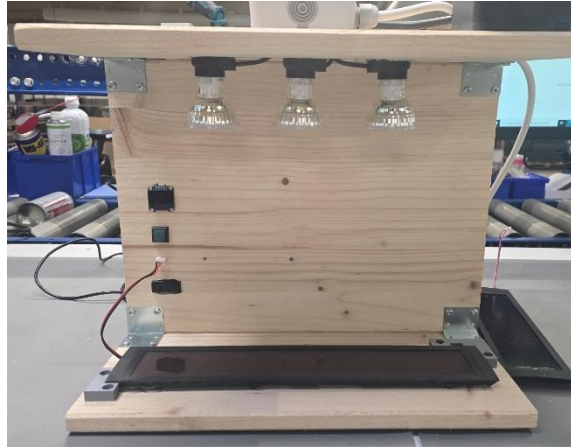


Figure 20 Banc de test automatisé et terminé

Une fois le banc de test terminé, j'ai rédigé la procédure d'utilisation pour que n'importe quel opérateur utilise le banc de test (voir procédure d'utilisation).

Le banc de test permet de réduire drastiquement le temps de test d'un panneau solaire d'environ 2 min à 30 secondes (tout dépend si le panneau solaire est défectueux ou encore en bon état). Il permet aussi une meilleure fiabilité, il n'y a plus de mesure manuelle à reporter sur un fichier qui peut entraîner des erreurs d'inattention.

Le nouveau banc de test permet aussi de compter le nombre de panneaux solaires testés dans la journée, ce qui lui permet de remplir son reporting plus simplement en fin de journée.

Le budget de ce nouveau banc de test s'élève à environ 150 €, ce qui est inférieur au budget de 500 € imposé par l'entreprise.

Les parties restantes à développer seraient la fabrication d'une boîte en impression 3D afin d'y placer les PCB et de les fixer correctement à l'arrière du banc de test. Il faudrait également concevoir des caches pour le contour de l'écran, du connecteur du panneau solaire et de l'interrupteur, car les trous utilisés pour l'intégration du PCB dans le banc de test sont visibles. Pour moi, il ne reste principalement que des améliorations esthétiques à apporter au banc de test actuel.

Pour améliorer le banc de test sur le plan fonctionnel, il serait possible de concevoir un PCB plus compact et mieux organisé, notamment en ajoutant des borniers pour l'écran, le bouton, l'interrupteur et le connecteur du panneau solaire. On pourrait également choisir un écran plus grand, capable d'afficher des statistiques telles que le nombre de panneaux solaires testés positivement sur le mois. Enfin, il serait possible d'ajouter un buzzer afin d'indiquer par un signal sonore la fin du test.

Ce projet d'automatisation de banc de test m'a apporté des compétences et de l'expérience en matière de conception de PCB. C'était la première fois que je concevais un circuit imprimé seul. Il m'a également permis d'améliorer mes compétences en programmation : j'ai appris à coder de manière autonome, à anticiper les fonctionnalités attendues et à concevoir un programme à partir de zéro. Ce projet m'a donc beaucoup apporté pour ma deuxième année de BTS et me sera utile pour la poursuite de mes études.

ATTESTATION SUR L'HONNEUR DE NON-PLAGIAT

Je soussigné, MALINGE Louison, candidat au Brevet de technicien supérieur Cybersécurité Informatique Électronique et Réseaux (BTS CIEL) certifie que le dossier technique intitulé « Projet Velux » est un travail personnel réalisé dans le cadre de ma formation.

Je déclare que ce document a été rédigé par mes soins et que toutes les sources, références, documents, ouvrages, sites internet ou travaux d'autres auteurs utilisés ont été clairement cités et référencés conformément aux règles en vigueur.

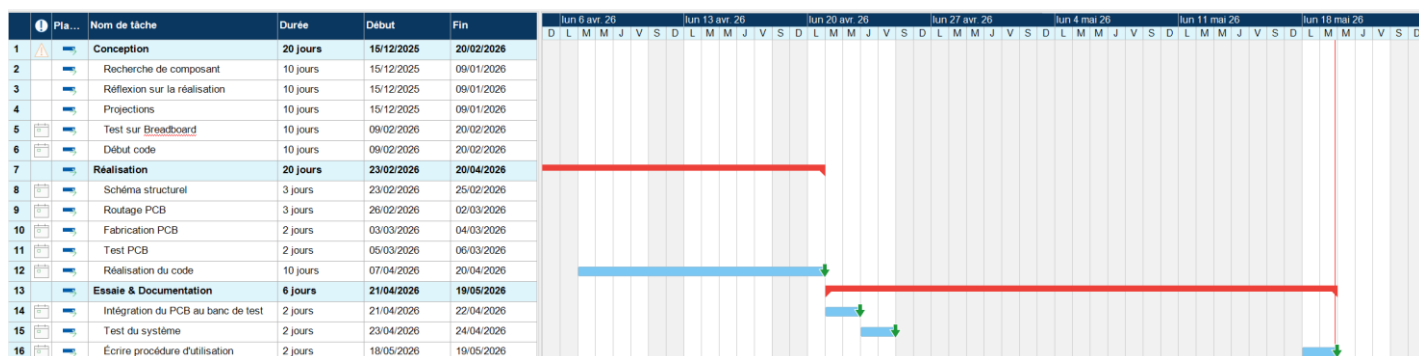
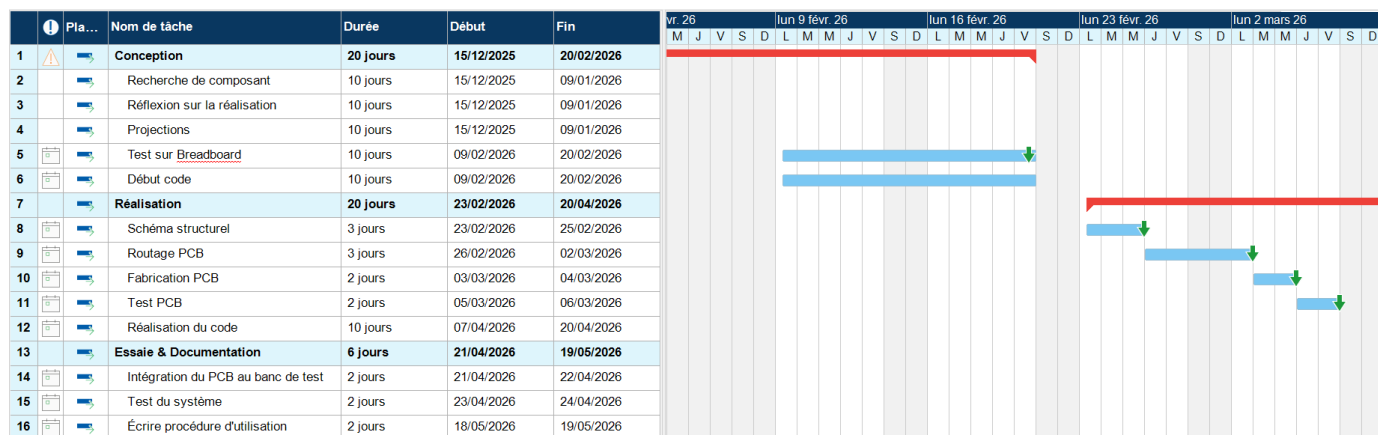
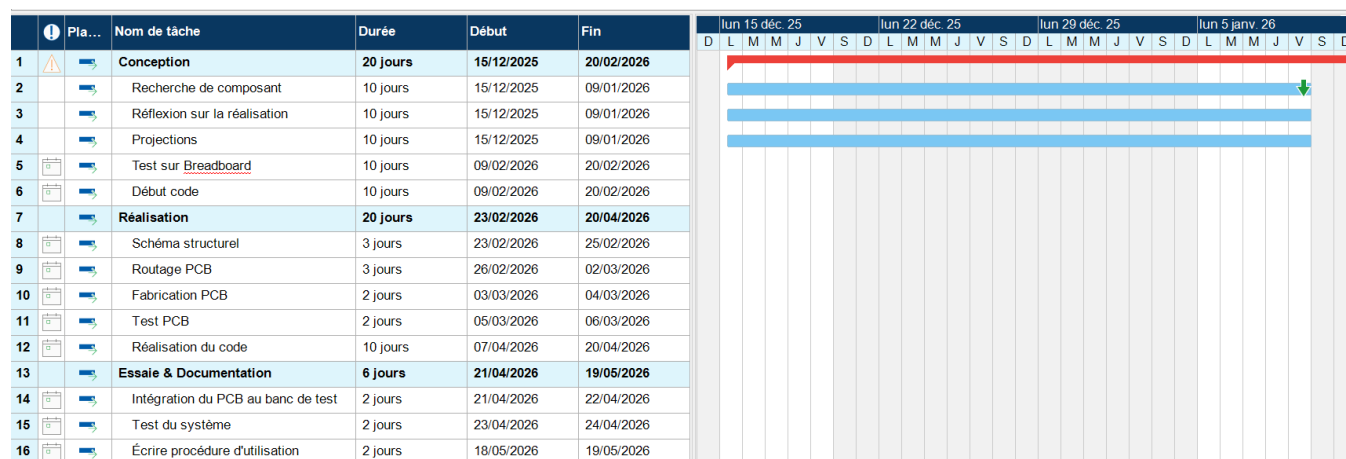
Je certifie également ne pas avoir eu recours au plagiat, à la copie partielle ou totale d'un travail existant, ni à une utilisation frauduleuse d'outils ou de contenus non signalés.

Fait à Valanjou 49670,

Le 22/05/2026,

Signature : Malinge

Annexe 1 : Planification Prévisionnelle



Annexe 2 : Planification réelle

